

Square Segmentation on BeagleBone AI-64 with TIDL 8.2 Setup Tutorial, Model Report, and Run Guide

Project documentation

June 9, 2026

Abstract

This document shows the BeagleBone AI-64 TIDL setup tutorial and the final square-segmentation model report. It covers flashing the correct image, validating the TIDL runtime, designing a TIDL 8.2-compatible ONNX graph, compiling with 08_02_00_01-rc1, deploying to the board, running inference through YAML files, and comparing CPU versus TIDL accuracy and latency. The selected model is a fully convolutional ONNX graph using only `Conv` and `Relu`.

The source repository is available at github.com/maria0419/beaglebone-ai-64-model-exemple. The repository keeps the BeagleBone/TIDL 8.2 compiler flow generic and places this square segmentation model under `models/square_seg_32x13/` as one example model.

1 Flash the Correct Image to eMMC

Use the official BeagleBoard image: [BeagleBone AI-64 Debian 11.7 2023-08-05 10GB Xfce eMMC Flasher](#).

BeagleBoard.org Debian Bullseye Xfce Image 2023-08-05

This image was selected because it is the most up-to-date image that already includes the BeagleBone AI-64 EdgeAI/TIDL runtime compatible with the TIDL 8.2 workflow. The eMMC flashing procedure was as follows:

1. Write the image to a microSD card.
2. Insert the microSD card into the BeagleBone AI-64.
3. Hold the boot button.
4. Power the board while holding the boot button during the start of boot.
5. Leave the board flashing the eMMC for about 15 minutes.
6. Power off the board, remove the microSD card, and boot again from eMMC.

After boot, the board responded over SSH at:

192.168.7.2 192.168.6.2

The default user for this image is:

user: debian password: temppwd

2 Board Runtime Check

Confirm ONNX Runtime and the TIDL providers on the board:

```
python3 -c "import onnxruntime as ort; print(ort.__version__); print(ort.get_available_providers())"
```

Expected for the image used in this project:

```
1.7.0  
['TIDLExecutionProvider', 'TIDLCompilationProvider', 'CPUExecutionProvider']
```

Confirm that the TIDL provider library exists:

```
find /usr -name 'libtidl_onnxrt_EP.so' 2>/dev/null
```

Expected:

```
/usr/lib/libtidl_onnxrt_EP.so
```

3 Board and Host Version Match

The TIDL compilation flow starts from the board runtime, not from the host. On the image selected in the flashing step, ONNX Runtime reported version 1.7.0 and these providers:

```
TIDLExecutionProvider  
TIDLCompilationProvider  
CPUExecutionProvider
```

Because the board runtime is fixed by that image, the host must compile the model with the matching TIDL 8.2 tools. The host-side stack used for this board image was:

```
edgeai-tidl-tools / tidl_tools 08_02_00_01-rc1  
onnxruntime_tidl 1.7.0  
Python 3.6 inside the compile container
```

3.1 Host Setup and Reproduction Flow

On a fresh Debian or Ubuntu host, install Docker first:

```
sudo apt update  
sudo apt install -y docker.io  
sudo systemctl enable --now docker  
sudo usermod -aG docker $USER
```

After adding the user to the `docker` group, log out and log in again so the new group membership is applied. Then verify Docker without `sudo`:

```
docker info
```

Download and extract the TIDL 8.2 host tools used by this project:

```
mkdir -p downloads tools/tidl82  
wget -O downloads/tidl_tools_08_02_00_01_rc1.tar.gz \  
    https://github.com/TexasInstruments/edgeai-tidl-tools/releases/download/08_02_00_01-rc1/  
    tidl_tools.tar.gz  
tar -xzf downloads/tidl_tools_08_02_00_01_rc1.tar.gz -C tools/tidl82
```

Verify that the extracted tools contain the ONNX Runtime TIDL provider libraries:

```
test -f tools/tidl82/tidl_tools/libtidl_onnxrt_EP.so
test -f tools/tidl82/tidl_tools/tidl_model_import_onnx.so
```

Build the TIDL 8.2 compile container once:

```
docker build -t tidl82-onnx-compiler:08_02_00_01_rc1 docker/tidl82
```

The container is needed because the `onnxruntime_tidl` wheel for this release targets Python 3.6. The wrapper itself is model-agnostic; the YAML file selects the already-exported ONNX model, the calibration images, and the output directory.

The final square segmentation artifacts are produced in this order:

1. Train the example with `python3 src/train.py -config configs/train.yaml`. This writes `artifacts/model.pt` and `artifacts/metrics.json`. The model itself is defined inside the `model: block` in `configs/train.yaml`; changing `model.name` and `model.params` changes which model is trained.
2. Re-export and compile with `./scripts/tidl82_export_compile.sh configs/compile_final.yaml`. This wrapper first exports `artifacts/model.onnx` from `artifacts/model.pt` on the host, then compiles that ONNX and writes `artifacts/tidl/model-artifacts/square\_seg/`.

For another ONNX model, copy `configs/compile_tidl82_template.yaml`, edit the ONNX path, calibration image directory, input size, and output directory, then pass that YAML to the same wrapper. If `calibration_dir` is an absolute host path, the wrapper mounts it read-only into the container automatically. For the square segmentation model, the one-command wrapper eventually calls the compiler script with the final compile preset inside the container:

```
python3 tools/compile_tidl.py --config models/square_seg_32x13/configs/compile.yaml
```

4 Dataset

The dataset contains grayscale TIFF image/mask pairs:

Table 1: Dataset and tensor format.

Item	Value
Training set	20,000 pairs
Test set	5,000 pairs
Input image	128 × 128, grayscale TIFF
Mask	binary, 0/255 TIFF
Input tensor	1 × 1 × 128 × 128, NCHW, float32 in [0, 1]
Output tensor	logits, 1 × 1 × 128 × 128
Post-processing	sigmoid outside ONNX, then threshold

5 Final Model

The selected architecture was not the highest floating-point model overall. It was the model that preserved the best mask quality after TIDL INT8 quantization on the real board.

Table 2: Final deployed model configuration.

Parameter	Value
Architecture	fully convolutional CNN
Hidden channels	32
Convolutional layers	13 total layers
Kernel size	3×3 for hidden layers
Final layer	1×1 convolution to one logit channel
Activations	ReLU only
Allowed ONNX ops	Conv, Relu
Export opset	11
TIDL tensor bits	8
Calibration frames	128
Measured CPU threshold	0.01
Measured TIDL threshold	0.99

The ONNX export check reported only **Conv** and **Relu**, with maximum absolute difference between PyTorch and ONNX Runtime CPU of 1.4496×10^{-4} on the export check tensor.

The exported ONNX file is:

```
artifacts/model.onnx
SHA256: 2ce83ab587844385590c5419eede2fb924a79099c064bc154399ee78e3eff328
```

The final TIDL artifact hashes are:

```
logits_tidl_net.bin
SHA256: 7cb39a232519374bbbe410ed441c8516acc8e6674c0de28c2c06076ffb5f95b7

logits_tidl_io_1.bin
SHA256: e79ce3232e2ffa511bc895e032e011cb0d52bfa051e632f53d387f05f43dfa76
```

6 Build a TIDL-Compatible ONNX Graph

The most important engineering constraint is not just training accuracy. The graph must be simple enough for TIDL 8.2 to compile and offload on the BeagleBone AI-64 runtime. The practical rules used were:

- use fixed input shape in NCHW format, here $1 \times 1 \times 128 \times 128$,
- keep the output fixed and leave post-processing outside the graph,
- export logits or raw tensors instead of embedding thresholding in ONNX,
- avoid dynamic shape logic,
- avoid operators that commonly break offload or force partial CPU fallback.

The safe starting operator set for this project was:

```
Conv
Relu
```

Operators intentionally avoided in the final ONNX graph were:

```
Resize
Upsample
ConvTranspose
Flatten
Gemm
MatMul
Sigmoid
Softmax
ArgMax
NonMaxSuppression
```

Sigmoid and threshold are applied in Python after inference. This keeps the model graph small and maximizes the chance that TIDL creates one fully offloaded subgraph.

7 Export ONNX with Opset 11

The export uses a fixed-shape dummy tensor and opset 11. For PyTorch versions with the newer exporter, the legacy exporter can be forced when needed:

```
torch.onnx.export(
    model,
    dummy_input,
    "artifacts/model.onnx",
    export_params=True,
    opset_version=11,
    do_constant_folding=True,
    input_names=["input"],
    output_names=["logits"],
    dynamic_axes=None,
    dynamo=False,
    external_data=False,
)
```

Validate the exported graph:

```
python3 -c "import onnx; m=onnx.load('artifacts/model.onnx'); onnx.checker.check_model(m);
    print([(o.domain,o.version) for o in m.opset_import]); print(sorted({n.op_type for n in
    m.graph.node}))"
```

The expected result for this project is:

```
[('', 11)]
['Conv', 'Relu']
```

A CPU ONNX Runtime smoke test should also be run on the host before TIDL compilation:

```
python3 -c "import numpy as np, onnxruntime as ort; x=np.zeros((1,1,128,128),dtype=np.
    float32); s=ort.InferenceSession('artifacts/model.onnx',providers=['
    CPUExecutionProvider']); y=s.run(None,{s.get_inputs()[0].name:x})[0]; print(y.shape, y.
    dtype)"
```

8 YAML Workflow

The workflow was simplified so that long command-line argument lists are no longer needed. Table 3 lists the YAML files in this example and what each one is for.

Table 3: YAML files used in the example workflow.

YAML file	What it is for
<code>configs/train.yaml</code>	Main training preset. It defines dataset paths, shared training settings, and the <code>model:</code> block used by the generic example.
<code>configs/train_32x13.yaml</code>	Candidate-specific training preset used to reproduce the selected 32x13 run.
<code>configs/export.yaml</code>	ONNX export preset. It reads <code>artifacts/model.pt</code> and writes <code>artifacts/model.onnx</code> .
<code>configs/compile.yaml</code>	TIDL compile preset. It selects the exported ONNX, calibration images, and the directory where the compiled TIDL artifacts are written.
<code>configs/run_cpu_board.yaml</code>	CPU inference preset for the packaged example layout.
<code>configs/run_tidl.yaml</code>	TIDL inference preset for the packaged example on the board.

9 Host-Side Commands

From the project directory, the full host-side flow for reproducing the final model is:

```
python3 src/train.py --config configs/train_32x13.yaml
./scripts/tidl82_export_compile.sh configs/compile.yaml
python3 src/run_tidl.py --config configs/run_cpu_board.yaml
```

The important dependency between these steps is that the compile YAML does not define the model architecture or training recipe. It only points the compiler to an already-exported ONNX file and to a representative calibration image set.

10 TIDL Compilation

The host-side TIDL compilation uses these effective settings:

Table 4: TIDL compilation settings.

Option	Value
Platform	J7
Version	8.2
Provider	TIDLCompilationProvider
Tensor bits	8
Accuracy level	1
Calibration frames	128
Max subgraphs	16

For this model, the one-command wrapper eventually calls:

```
python3 tools/compile_tidl.py --config models/square_seg_32x13/configs/compile.yaml
```

The one-command wrapper first exports the ONNX model on the host and then launches the TIDL compile step. The compile stage still consumes an ONNX model that was already exported; it does not recreate the training run or choose the saved model state by itself.

The effective provider stack during compilation is:

```
providers = ['TIDLCompilationProvider', 'CPUExecutionProvider']
```

The important provider options are:

```
delegate_options = {
    'tidl_tools_path': '/opt/tidl_tools',
    'artifacts_folder': 'artifacts/tidl/model-artifacts/square_seg',
    'platform': 'J7',
    'version': '8.2',
    'tensor_bits': 8,
    'debug_level': 1,
    'max_num_subgraphs': 16,
    'deny_list': '',
    'accuracy_level': 1,
    'advanced_options:calibration_frames': 128,
    'advanced_options:calibration_iterations': 5,
    'advanced_options:add_data_convert_ops': 3,
    'advanced_options:quantization_scale_type': 0,
    'advanced_options:high_resolution_optimization': 0,
    'advanced_options:pre_batchnorm_fold': 1,
    'ti_internal_nc_flag': 1601,
}
```

The compiler reported:

```
Final number of subgraphs created are : 1, - Offloaded Nodes - 25, Total Nodes - 25
ALL MODEL CHECK PASSED
```

The compiler also emitted a MSMC warning from `ti_cnnperfsim.out`. In this project the warning did not block runtime execution on the real BeagleBone AI-64, but it should still be tracked as a deployment risk.

11 Final Deploy Package

The final deploy package is:

```
artifacts/square_seg_32x13_tidl82_final.tar.gz
```

Its relevant content is:

```
model.onnx
run_tidl.py
config_utils.py
configs/run_cpu_board.yaml
configs/run_tidl.yaml
test_images/img_0001_image.tif
artifacts/tidl/model-artifacts/square_seg/*
```

The final local TIDL artifact folder is:

```
artifacts/tidl/model-artifacts/square_seg_32x13_final
```

Copy and extract the package on the board:

```
scp artifacts/square_seg_32x13_tidl82_final.tar.gz debian@192.168.7.2:/home/debian/
ssh debian@192.168.7.2
cd /home/debian
rm -rf square_seg_tidl_final_yaml
mkdir -p square_seg_tidl_final_yaml
tar -xzf square_seg_32x13_tidl82_final.tar.gz -C square_seg_tidl_final_yaml
cd square_seg_tidl_final_yaml
```

The archive extraction can emit timestamp warnings when the board clock is behind the host clock. Those warnings do not affect inference.

12 Board-Side Commands

After copying or extracting the final package on the board, the short commands inside the deployed package are:

```
python3 run_tidl.py --config configs/run_cpu_board.yaml
sudo python3 run_tidl.py --config configs/run_tidl.yaml
```

These exact YAML-based commands were tested successfully on the BeagleBone AI-64. The TIDL run log contained:

```
libtidl_onnxrt_EP loaded
Final number of subgraphs created are : 1, - Offloaded Nodes - 25, Total Nodes - 25
```

13 Measured CPU versus TIDL Results

All board measurements in this section use `img_0001_image.tif`, the corresponding binary label, and the same exported ONNX model.

Table 5: Board accuracy and latency for the final 32x13 model.

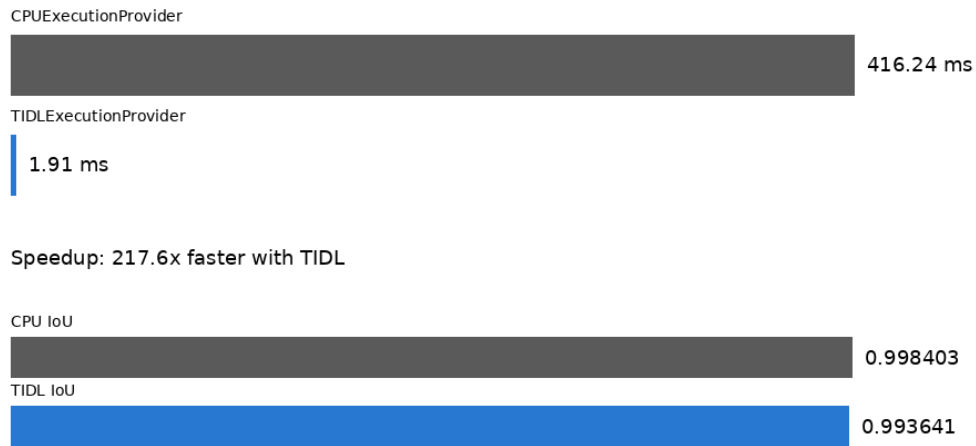
Provider	Threshold	IoU	Dice	Mean time	Median time
CPUExecutionProvider	0.01	0.998403	0.999200	416.24 ms	416.33 ms
TIDLExecutionProvider	0.99	0.993641	0.996808	1.91 ms	1.98 ms

The measured speedup is approximately:

$$\frac{416.24 \text{ ms}}{1.91 \text{ ms}} \approx 217.6\times$$

The TIDL benchmark data for the measured run indicates roughly 1.43 ms of subgraph processing time between `ts:subgraph_logits_proc_start` and `ts:subgraph_logits_proc_end`. The remaining wall time comes from copies and runtime overhead.

BeagleBone AI-64: CPU vs TIDLExecutionProvider



Measured on img_0001, 128x128 grayscale segmentation. CPU: 50 runs. TIDL: 20 runs. Thresholds tuned per provider.

Figure 1: CPU versus TIDL latency and IoU on the measured BeagleBone AI-64 sample.

14 Qualitative Result

Figure 2 compares the input image, ground-truth mask, CPU mask, TIDL mask, and a TIDL error map. In the error map, green pixels are true positives, red pixels are false positives, and blue pixels are false negatives.

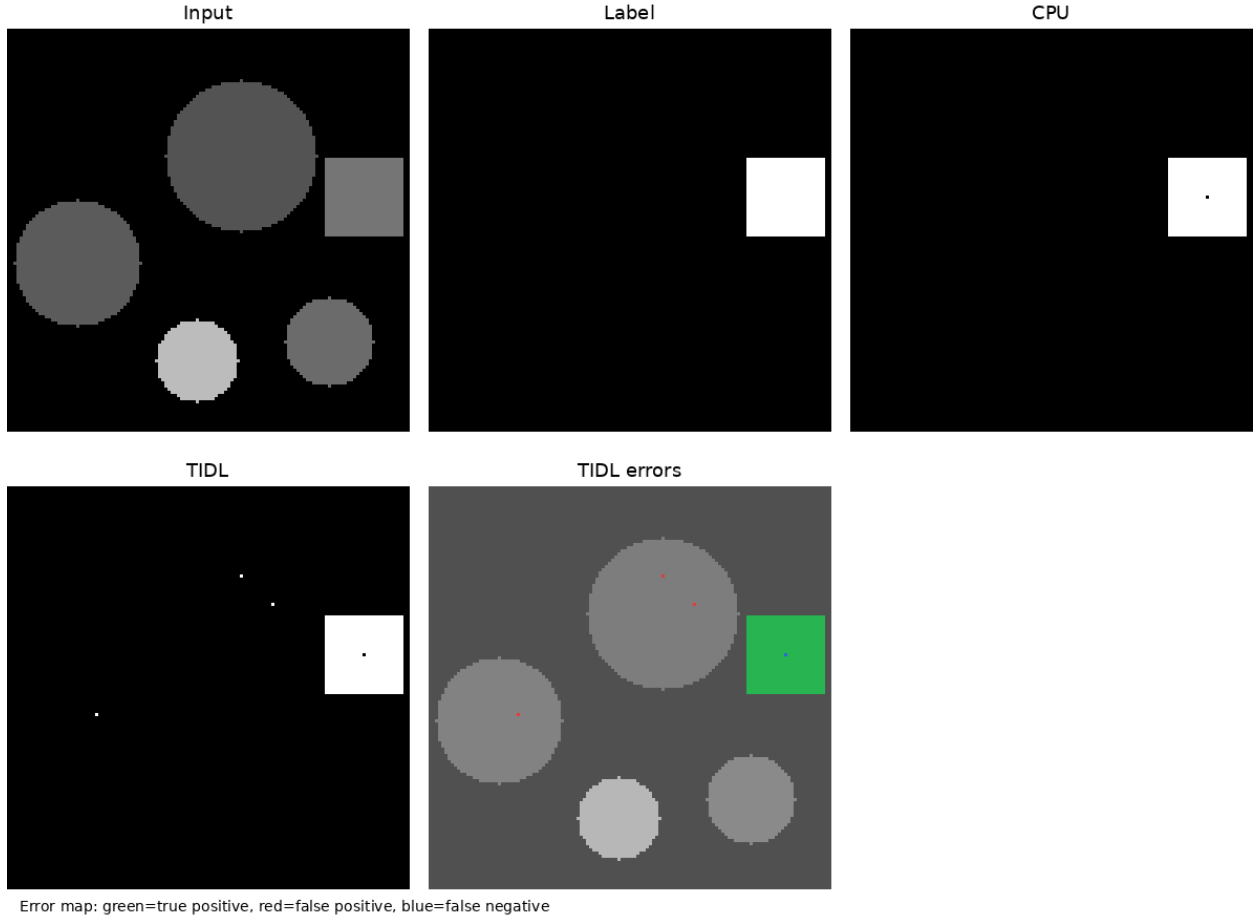


Figure 2: Qualitative comparison of CPU and TIDL outputs for the final model.

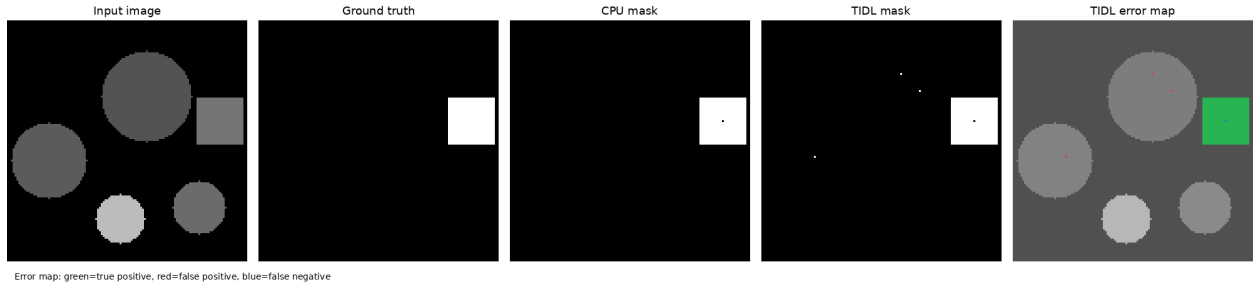


Figure 3: Wide qualitative view with the same CPU and TIDL masks.

15 Why This Model Was Chosen

The best floating-point model was not the best deployed TIDL model. The final decision was made using the actual TIDL output on the board.

Table 6: Candidate comparison on the same board sample after TIDL quantization.

Candidate	Offloaded nodes	Best TIDL threshold	TIDL IoU
32 channels, 15 layers, INT8	29	0.50	0.913738
32 channels, 15 layers, logit L2	29	0.75	0.953822
32 channels, 13 layers, final	25	0.99	0.993641

TIDL candidate comparison on board sample



The final model sacrifices some float-test IoU but survives INT8 quantization much better on TIDL.

Figure 4: TIDL IoU comparison among tested model candidates.

16 Recommended Usage

If the goal is to reproduce the final deployed model from scratch, use this order:

1. On the host, train the selected candidate:

```
python3 src/train.py --config configs/train_32x13.yaml
```

2. On the host, export the trained model state to ONNX:

```
python3 src/export_onnx.py --config configs/export.yaml
```

3. On the host, compile the exported ONNX into TIDL artifacts:

```
./scripts/tidl82_onnx_compile.sh configs/compile.yaml
```

4. Deploy `artifacts/square_seg_32x13_tidl82_final.tar.gz` to the board.

5. On the board, run CPU inference:

```
python3 run_tidl.py --config configs/run_cpu_board.yaml
```

6. On the board, run TIDL inference:

```
sudo python3 run_tidl.py --config configs/run_tidl.yaml
```

If you are reusing the already released final model state and ONNX file, the training step can be skipped and the workflow starts at export or compile, depending on which artifacts you already have.

17 Conclusion

The final 32-channel, 13-layer convolutional model is the best current deployment choice because it preserves high mask accuracy after TIDL INT8 quantization while keeping full graph offload. The measured TIDL path is about $217.6\times$ faster than CPU ONNX Runtime on the BeagleBone AI-64 for the measured sample, while the IoU remains very close to the CPU reference.